SIMATS ENGINEERING

JAVA PROGRAMMING — ASSIGNMENT 1

Campus Smart-Parking System

PART 1

Student: Doma Saketh

Register Number: 192525018

Course: Java Programming

Total Assignment Marks: 50

GitHub repository

https://github.com/sakethdoma61-11/CSA09-192525018-Assignment1/tree/main

Repository link supplied by student; verify report examples against submitted source files.

Contents

Task 1 — Problem Analysis and Objects

Task 2 — Data Representation

Task 3 — Control Logic and Pseudocode

Task 4 — Classes, Objects and Constructors

Task 5 — Static Members and Object Life Cycle

Task 6 — Inheritance Hierarchy

Task 1 — Understand the Problem and Identify the Objects (4 marks)

The system manages multi-floor campus parking for two-wheelers, cars and electric vehicles (EVs). On entry it allocates a suitable free slot, issues a unique ticket and records entry time. On exit it releases the slot and calculates the fee. Staff and students with valid monthly passes park free; visitors pay the applicable fee. EV charging may be billed separately.

Inputs: registration number, vehicle type, pass status, entry/exit time, floor capacity and charging usage. **Outputs:** slot assignment, ticket number, fee, occupancy status and daily collection.

Candidate classes and responsibilities

Class	Responsibility
Vehicle (abstract)	Common registration identity and vehicle behaviour
TwoWheeler / Car / ElectricCar	Vehicle-specific rate or charging capability
ParkingFacility	Holds floor/slot state; allocates and releases slots
ParkingTicket	Unique ID, vehicle, entry time, exit and fee
ParkingPass	Represents monthly or one-day parking entitlement
PricingRule	Calculates fees according to a configurable rule
ParkingService	Coordinates entry, exit, ticket lookup and validation

Methods: allocateSlot(), releaseSlot(), issueTicket(), findTicket(), calculateFee(), recordExit() and dailyCollection(). **Business rules:** one vehicle per slot; no duplicate active registration; ticket IDs are not reused; invalid coordinates are rejected; fees cannot be negative; releasing an empty slot fails; a full facility reports no availability. Assume registration numbers uniquely identify active vehicles.

Objects versus parallel arrays: objects keep data and behaviour together. Parallel arrays such as regNo[], type[] and entryTime[] require matching indexes and repeated updates when a new attribute is added. Searching an unsorted collection is O(n) in either design; objects are easier to extend and validate.

Task 2 — Choose the Data Representation (3 marks)

Use String for registration numbers, int for floor/slot and duration in minutes, long for timestamps, double or preferably BigDecimal for money, and boolean for flags. Use Scanner for console input. A jagged array String[][] is appropriate because floors can have different capacities; each row is separately allocated.

```
String[][] slots = new String[slotsPerFloor.length][];
for (int f = 0; f < slotsPerFloor.length; f++)
    slots[f] = new String[slotsPerFloor[f]];
```

Integer division truncates: 61 / 60 equals 1, which undercharges if every started hour is billed. Ceiling division for non-negative minutes avoids this: hours = (minutes + 59) / 60. Reject negative durations and handle zero explicitly.

```
int billableHours(int minutes) {
    if (minutes <= 0) return 0;
    return (minutes + 59) / 60;
}
```

For S total slots and F floors, storage is O(S + F). A linear scan takes O(S) worst-case. A flat array index based on floor × slotsPerFloor only works directly when each floor has equal capacity; unequal floors require offsets or a jagged structure. ArrayList also supports variable row lengths.

Task 3 — Design the Control Logic (4 marks)

Allocation pseudocode: scan floors in order; scan slots on each floor; when a slot is empty, store the registration and return its floor/slot; if none is free, return null. **Release pseudocode:** validate floor and slot bounds; if invalid or empty, return false; otherwise clear the slot and return true. **Fee pseudocode:** validate duration; waive fee for an eligible pass; compute billable hours; multiply by rate; add charging fee if applicable; apply any configured daily cap.

Completed supplied ParkingFacility program

```
import java.util.Arrays;
public class ParkingFacility {
    private final String[][] slots;
```

```
    public ParkingFacility(int[] slotsPerFloor) {
        slots = new String[slotsPerFloor.length][];
        for (int f = 0; f < slotsPerFloor.length; f++)
            slots[f] = new String[slotsPerFloor[f]];
    }
    public int[] allocate(String vehicleNo) {
        if (vehicleNo == null || vehicleNo.isBlank()) return null;
        for (int f = 0; f < slots.length; f++)
            for (int s = 0; s < slots[f].length; s++)
                if (slots[f][s] == null) {
                    slots[f][s] = vehicleNo;
                    return new int[]{f, s};
                }
        return null;
    }
    public boolean release(int floor, int slot) {
        if (floor < 0 || floor >= slots.length) return false;
        if (slot < 0 || slot >= slots[floor].length) return false;
        if (slots[floor][slot] == null) return false;
        slots[floor][slot] = null;
        return true;
    }
    public static void main(String[] args) {
        ParkingFacility p = new ParkingFacility(new int[]{2, 1});
        System.out.println(Arrays.toString(p.allocate("TN01AB1111")));
        System.out.println(Arrays.toString(p.allocate("TN02CD2222")));
        System.out.println(Arrays.toString(p.allocate("TN03EF3333")));
        System.out.println(Arrays.toString(p.allocate("TN04GH4444")));
        System.out.println(p.release(0,1)+" "+p.release(0,1)+" "+p.release(5,0));
        System.out.println(Arrays.toString(p.allocate("TN04GH4444")));
    }
}
```

Expected output specified in the assignment

```
[0, 0]
[0, 1]
[1, 0]
null
true false false
[0, 1]
```

Allocation is O(S) worst-case; release is O(1) after bounds validation. A stack of free-slot coordinates could make allocation/release approximately O(1), but requires extra storage and correct synchronization of the stack with occupancy.

Task 4 — Classes, Objects and Constructors (4 marks)

Constructors establish valid state. A default constructor takes no arguments; a parameterized constructor receives required values; overloaded constructors offer supported initialization paths. Use this.field to distinguish fields from parameters and this(...) to chain constructors. A copy constructor is only needed when an explicit copy is meaningful; duplicating a ticket should not duplicate its unique ID.

```
class ParkingTicket {
    private final long ticketId;
    private final String registration;
    private final long entryMillis;
    ParkingTicket(long id, String reg) {
        this(id, reg, System.currentTimeMillis());
    }
    ParkingTicket(long id, String reg, long time) {
        if (reg == null || reg.isBlank())
            throw new IllegalArgumentException("Registration required");
        if (time < 0) throw new IllegalArgumentException("Invalid time");
        ticketId = id; registration = reg; entryMillis = time;
    }
    public long getTicketId() { return ticketId; }
    public String getRegistration() { return registration; }
    public long getEntryMillis() { return entryMillis; }
}
```

In the usual JVM model, local references are in stack frames and objects/arrays are heap allocated; references point to objects. Creating n tickets takes O(n) total time and O(n) heap space. Setter-based initialization can expose invalid intermediate states; constructors or validated factories reduce that risk.

Task 5 — Static Members and Object Life Cycle (3 marks)

A static ticket counter is shared by all instances. If north and south gates each use a separate instance counter, they can issue duplicate IDs; use a shared counter or prefix IDs with the facility identifier. Static methods suit operations that do not depend on instance state. After exit, remove a ticket from active storage; if no references remain, it becomes eligible for garbage collection, though collection timing is not guaranteed.

```
class TicketIds {
    private static long nextId = 1;
    private TicketIds() {}
```

```
    public static synchronized long next() { return nextId++; }
}
```

Java passes object references by value: a method can mutate the referenced object, but reassigning its local reference does not change the caller's reference.

Task 6 — Design the Inheritance Hierarchy (4 marks)

Use abstract Vehicle as a superclass, with TwoWheeler and Car subclasses; ElectricCar may extend Car and implement Chargeable. This is hierarchical inheritance from Vehicle and multilevel inheritance for ElectricCar. The base class stores common registration data and declares ratePerHour().

```
abstract class Vehicle {
    private final String regNo;
    protected Vehicle(String regNo) { this.regNo = regNo; }
    public String getRegNo() { return regNo; }
    protected abstract double ratePerHour();
}
class Car extends Vehicle {
    Car(String regNo) { super(regNo); }
    @Override protected double ratePerHour() { return 30.0; }
}
interface Chargeable { double chargingFee(int kWh); }
class ElectricCar extends Car implements Chargeable {
    ElectricCar(String regNo) { super(regNo); }
    @Override public double chargingFee(int kWh) {
        if (kWh < 0) throw new IllegalArgumentException("Negative kWh");
        return 12.0 * kWh;
    }
}
```

Constructor trace: ElectricCar constructor calls Car constructor, which calls Vehicle constructor; Vehicle initializes the registration, then control returns to Car and ElectricCar. UML summary: TwoWheeler extends Vehicle; Car extends Vehicle; ElectricCar extends Car and implements Chargeable.

Inheritance versus composition: inheritance models “is-a” and enables overriding. Composition stores a VehicleType or PricingRule inside a vehicle and is more flexible when rates change independently of category. Fee calculation is O(1) in either design; inheritance usually adds a subclass for each new type, while composition may allow adding configuration instead.

JAVA ASSIGNMENT 1 — PART 1

Task01 — execution output

Java terminal output — Task01

```
Command: javac *.java  |  java Task01Demo

> Registration: TN01AB1111
> Vehicle type: Car
> Candidate classes: Vehicle, ParkingFacility, ParkingTicket
> Rule check: one vehicle represented by one object
```

Output captured from a successful local compile/run of the included demo source.

Demo-program output; not a screenshot from the student's own computer.

JAVA ASSIGNMENT 1 — PART 1

Task02 — execution output



Demo-program output; not a screenshot from the student's own computer.

JAVA ASSIGNMENT 1 — PART 1

Task03 — execution output

Java terminal output — Task03

```
Command: javac *.java  |  java ParkingFacility

> [0, 0]
> [0, 1]
> [1, 0]
> null
> true false false
> [0, 1]
```

Output captured from a successful local compile/run of the included demo source.

Demo-program output; not a screenshot from the student's own computer.

JAVA ASSIGNMENT 1 — PART 1

Task04 — execution output



Demo-program output; not a screenshot from the student's own computer.

JAVA ASSIGNMENT 1 — PART 1

Task05 — execution output



Demo-program output; not a screenshot from the student's own computer.

JAVA ASSIGNMENT 1 — PART 1

Task06 — execution output



Demo-program output; not a screenshot from the student's own computer.